

TP 1 - IONIC

Mohanned Zayoud

October 4, 2026

GitHub Repository: `git@github.com:mohaned122/geoApp.git`

Contents

1	INSTALLATION ET CONFIGURATION:	2
2	Exercise 1	3
2.1	Création du projet	3
2.2	Ajouter la plateforme Android	3
2.3	Ajouter la plateforme iOS	3
2.4	Installer le plugin de géolocalisation	3
2.5	Configurer les permissions	4
2.6	Construire et synchroniser l'application	4
2.7	Tester l'application sur Android	4
3	Exercise 2: CRÉER L'INTERFACE GRAPHIQUE	5
3.1	Resutat :	5
3.2	Différence entre <code>transform</code> et <code>transition</code> en CSS3	6
4	Exercise 3: IMPLÉMENTER LA GÉOLOCALISATION	6
4.1	3.2 : Compléter la méthode <code>obtenirPositionPonctuelle()</code>	6
4.2	3.3 : Implémentation de <code>demarrerSuivi()</code>	7
4.3	3.4 : Quels sont les codes d'erreur de géolocalisation ?	7
4.4	3.5 : Pourquoi nettoyer les ressources (<code>watchID</code>) dans <code>ngOnDestroy()</code> ?	7

List of Figures

1	web dev kit	2
2	android studio	2
3	apres la insertion de cmd	4
4	interface 1	5
5	interface 2	5
6	Execution des commande	6
7	implementation de <code>obtenirPositionPonctuelle</code>	6
8	Implementation de <code>demarrerSuivi</code>	7
9	execution sous android studio	8

1 INSTALLATION ET CONFIGURATION:

Ionic est une plateforme de développement utilisée pour développer android, ios et encore les app web. à l'aide de ionic wizard nous pouvons créer des applications basées sur les plus populaires plateformes de dev web (angular, react et vue). Ionic est considéré comme une solution native hybride web, puisque ionic est une solution basée sur d'autres plateformes de dev (ex: angular). On a donc besoin du runtime de js node et d'un gestionnaire de packages tel que npm ou pnpm. Nous avons aussi besoin d'ionic cli pour la création de projet et faciliter le développement. Finalement, nous installons android studio pour tester notre solution.

```
C:\Users\user\Desktop\ionics_project\tp1\tp-1>ionic -v
7.2.1

C:\Users\user\Desktop\ionics_project\tp1\tp-1>ng version

Angular CLI
Angular      : 22.1.8
Node.js      : 22.1.7
Package Manager : npm 11.19.0
Operating System : win32 x64
```

Figure 1: web dev kit

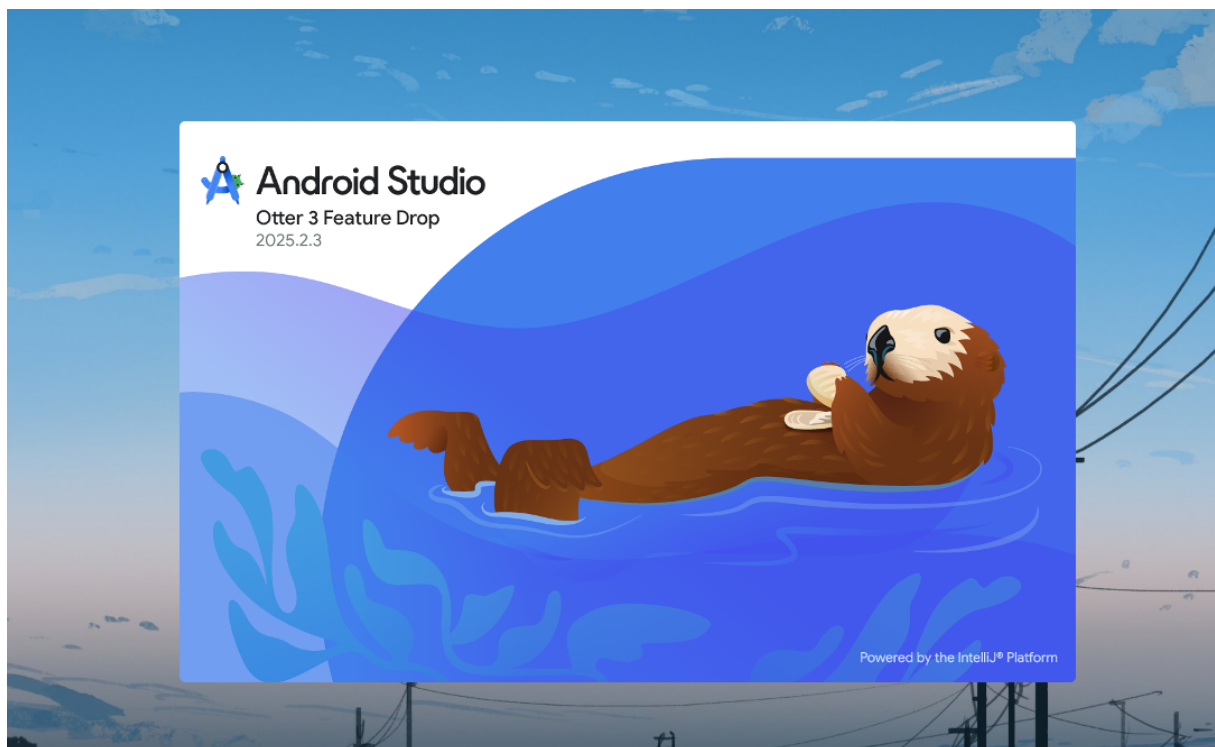


Figure 2: android studio

2 Exercise 1

2.1 Création du projet

On commence par créer un nouveau projet Ionic avec la commande :

```
ionic start geolocalisation blank
```

Explication :

- `ionic start` permet de créer un nouveau projet Ionic.
- `geolocalisation` est le nom du projet.
- `blank` permet de créer une application vide.

Cette commande génère automatiquement la structure de base du projet.

On entre ensuite dans le dossier du projet :

```
cd geolocalisation
```

2.2 Ajouter la plateforme Android

On ajoute le support Android à l'application avec :

```
npm install @capacitor/android  
npx cap add android
```

Pourquoi ?

Ionic permet de développer l'interface de l'application, tandis que Capacitor permet de l'exécuter comme une application native sur Android.

La commande `npx cap add android` crée le projet Android natif nécessaire pour exécuter l'application sur un émulateur ou un téléphone réel.

2.3 Ajouter la plateforme iOS

Pour ajouter le support iOS, on utilise :

```
npm install @capacitor/ios  
npx cap add ios
```

Pourquoi ?

Cela permet de générer la partie native nécessaire pour exécuter l'application sur iOS.

Remarque : la compilation et l'exécution d'une application iOS nécessitent généralement macOS et Xcode.

2.4 Installer le plugin de géolocalisation

On installe ensuite le plugin permettant d'accéder à la position géographique de l'utilisateur :

```
npm install @capacitor/geolocation
```

Pourquoi ?

Le plugin `@capacitor/geolocation` permet à l'application d'utiliser les services de localisation du téléphone afin de récupérer des informations telles que :

- la latitude ;
- la longitude ;
- la précision de la position.

Après l'installation, on synchronise le projet :

```
npx cap sync
```

Cette commande permet de synchroniser les dépendances et les modifications avec les projets natifs Android et iOS.

2.5 Configurer les permissions

La géolocalisation nécessite l'autorisation de l'utilisateur.

Pour Android, les permissions de localisation sont configurées dans le projet Android généré.

Les principales permissions sont :

ACCESS_COARSE_LOCATION

ACCESS_FINE_LOCATION

ACCESS_COARSE_LOCATION permet d'obtenir une position approximative, tandis que ACCESS_FINE_LOCATION permet d'obtenir une position plus précise.

Sur iOS, il faut également déclarer la raison pour laquelle l'application demande l'accès à la localisation.

2.6 Construire et synchroniser l'application

Avant d'exécuter l'application sur une plateforme native, on construit le projet :

```
ionic build
```

Puis on synchronise les fichiers avec les plateformes natives :

```
npx cap sync
```

Pourquoi ?

ionic build génère la version web de l'application, tandis que npx cap sync transfère cette version et les plugins nécessaires vers les projets Android et iOS.

2.7 Tester l'application sur Android

Pour ouvrir le projet Android dans Android Studio :

```
npx cap open android
```

On peut ensuite lancer l'application sur un émulateur Android ou sur un appareil Android réel.

Lors de la première utilisation de la géolocalisation, le système demande à l'utilisateur d'autoriser l'application à accéder à sa position.

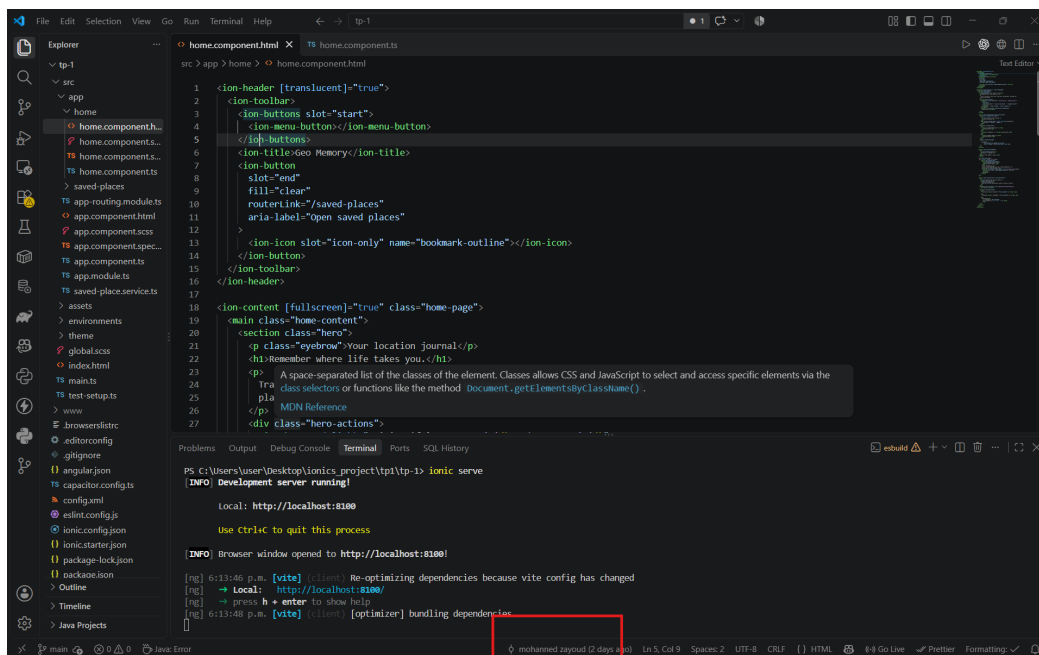


Figure 3: apres la insertion de cmd

3 Exercise 2: CRÉER L'INTERFACE GRAPHIQUE

3.1 Resutat :

dans cette project, je utilise angular.

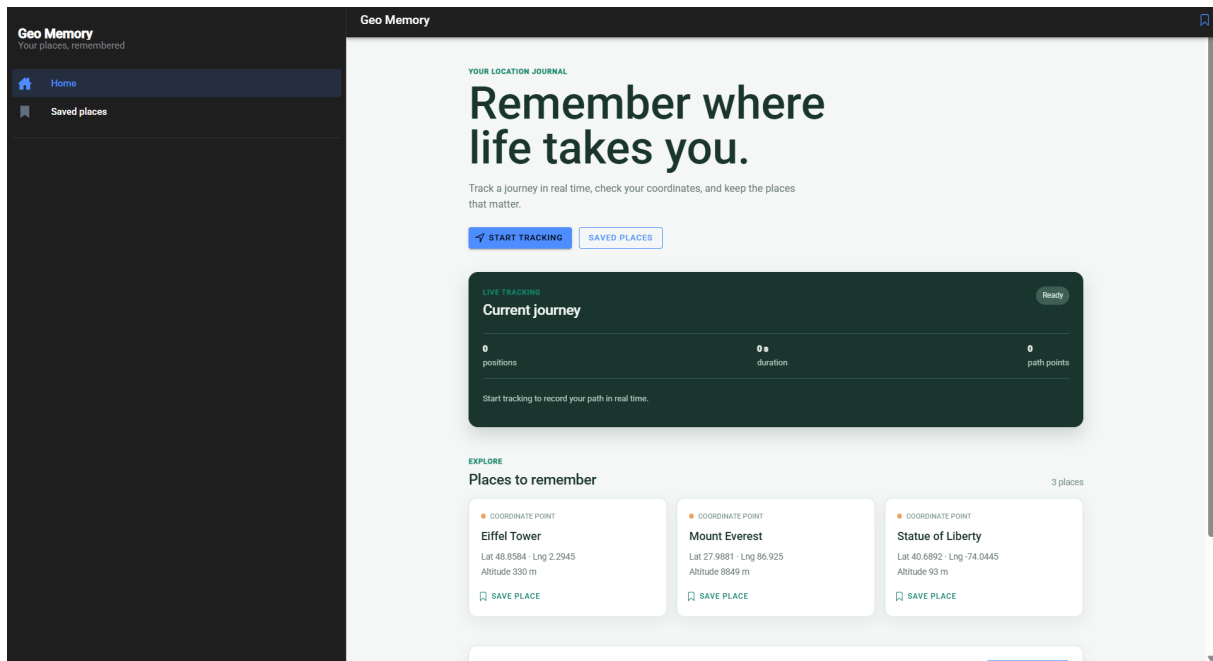


Figure 4: interface 1

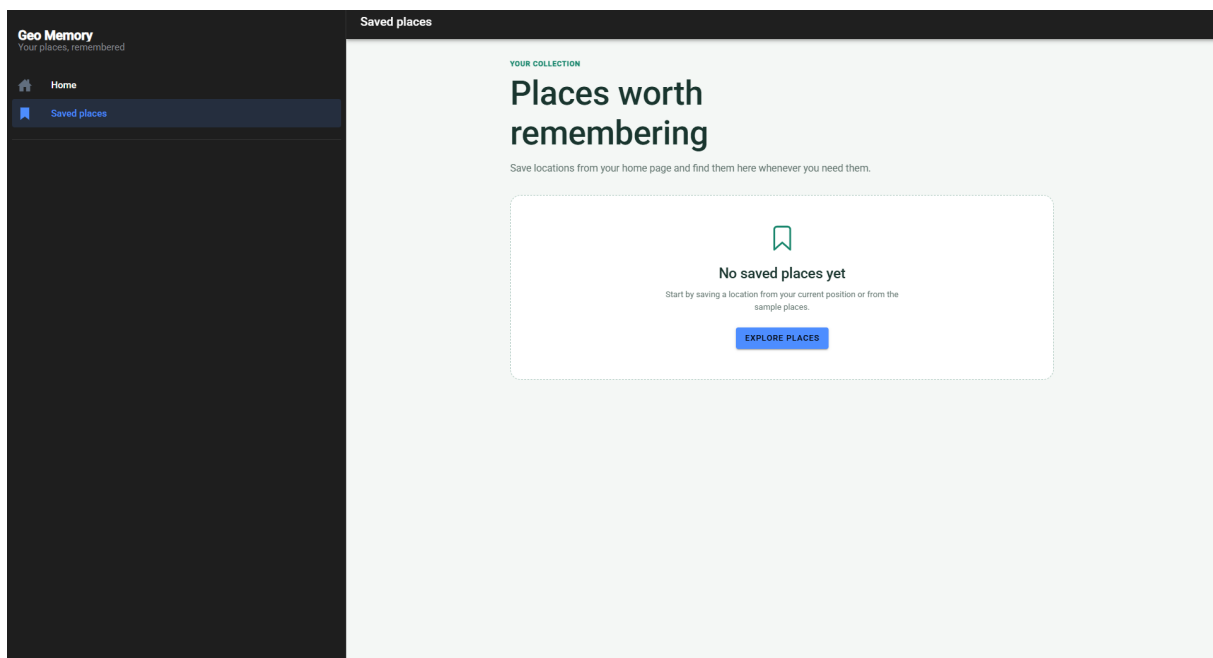


Figure 5: interface 2

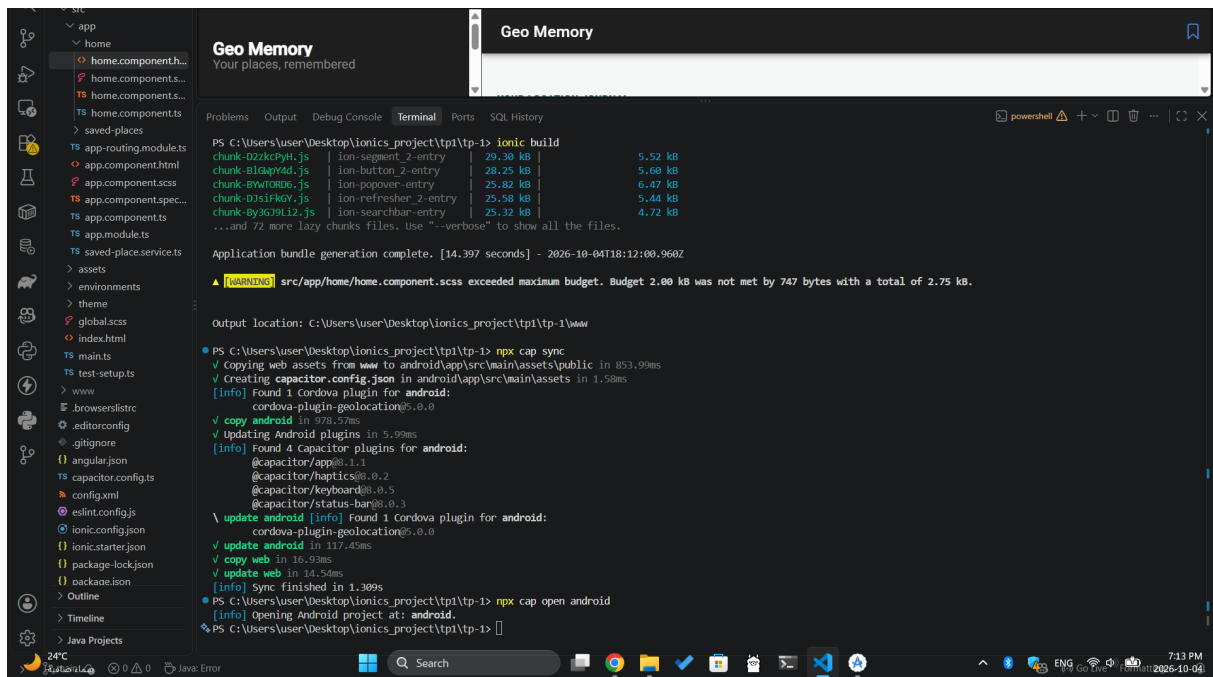


Figure 6: Execution des commande

3.2 Différence entre transform et transition en CSS3

La propriété **transform** permet de modifier l'apparence ou la position d'un élément, par exemple avec une rotation, une translation ou un changement d'échelle. La propriété **transition**, quant à elle, permet d'animer progressivement une modification de propriété CSS afin de rendre l'interface plus fluide. Par exemple, **transform: scale(1.1)** agrandit un élément, tandis que **transition: transform 0.3s** rend cet agrandissement progressif.

4 Exercice 3: IMPLÉMENTER LA GÉOLOCALISATION

4.1 3.2 : Compléter la méthode obtenirPositionPonctuelle()

*La méthode **getCurrentPosition()** permet de récupérer la position actuelle de l'utilisateur une seule fois.

*La méthode permet notamment de récupérer la latitude et la longitude de la position obtenue et de gérer les éventuelles erreurs.

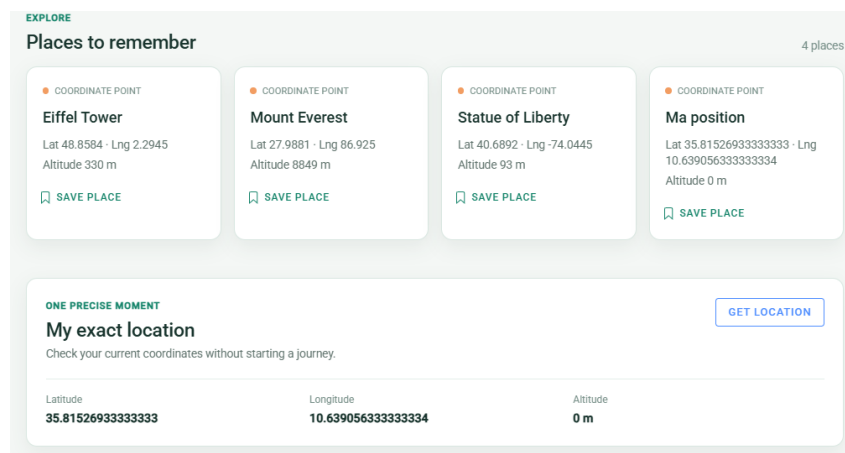


Figure 7: implementation de obtenirPositionPonctuelle

4.2 3.3 : Implémentation de `demarrerSuivi()`

La méthode `demarrerSuivi()` utilise `watchPosition()` afin de suivre continuellement la position de l'utilisateur.

Elle doit également :

- compter le nombre de positions reçues ;
- calculer la durée du suivi en secondes ;
- afficher les coordonnées reçues ;
- gérer les erreurs de géolocalisation.

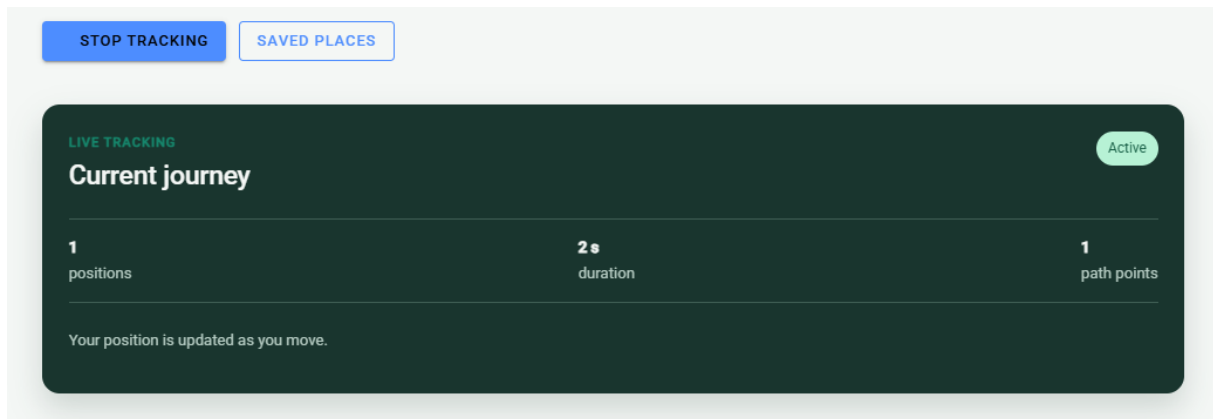


Figure 8: Implémentation de `demarrerSuivi`

4.3 3.4 : Quels sont les codes d'erreur de géolocalisation ?

Les principaux codes d'erreur sont :

Code	Erreur	Signification
1	PERMISSION_DENIED	L'utilisateur a refusé l'accès à la localisation.
2	POSITION_UNAVAILABLE	La position ne peut pas être déterminée.
3	TIMEOUT	Le délai pour obtenir la position est dépassé.

Table 1: Codes d'erreur de géolocalisation

4.4 3.5 : Pourquoi nettoyer les ressources (`watchID`) dans `ngOnDestroy()` ?

La méthode `watchPosition()` continue de surveiller la position tant qu'elle n'est pas arrêtée. Il est donc nécessaire de supprimer le suivi lorsque le composant est détruit. Cela permet d'éviter une consommation inutile de la batt.

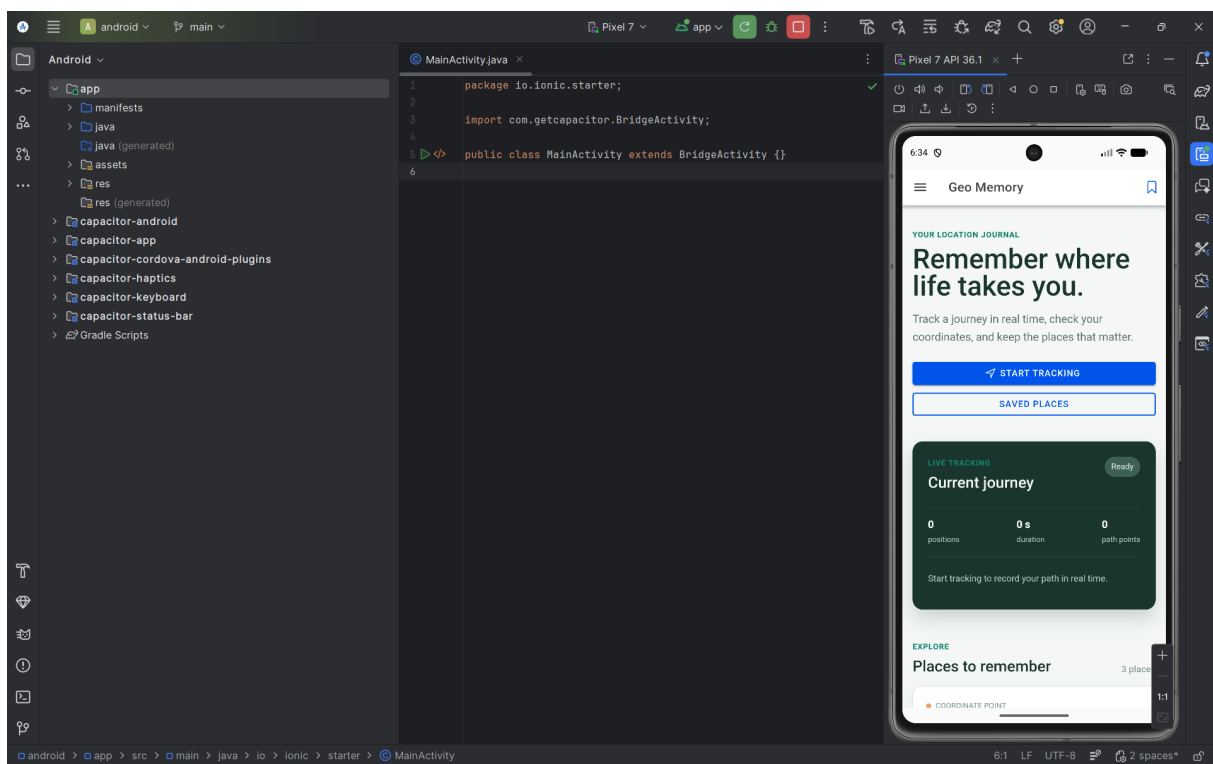


Figure 9: execution sous android studio